



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing



Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py`.
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space is the set of all possible states which could be achieved by applying actions to the initial state. For example, in the case of Grid World environment, each position which is possible for the agent to go can be considered as the state, and actions like up, down, left, and right can be used to connect these states.

Search Tree, however, is the tree-like structure formed by the search algorithm during the process of searching. In Search Tree, the root corresponds to the initial state and child nodes correspond to the possible successors of the initial state. It is quite possible that the same state occurs several times in a search tree depending on the paths.

Thus, State Space refers to the possible states of the actual problem whereas the Search Tree refers to the way the search algorithm searches the states.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set keeps the memory of the states visited during search by the search algorithm so that the same state will not be expanded multiple times. This would cause a cyclic behavior if the reached set is not considered, as the agent keeps moving in the same set of states over and over. It becomes even more relevant in the case of DFS as it always moves deeper in the available path rather than searching elsewhere in other paths.

So this causes the problem where DFS may get stuck moving deeper along a cycle or unnecessary path rather than reaching the goal point. Hence, with the help of reached sets, the search becomes a graph search problem preventing the issue of re-visiting the same state.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

It always expands the shallower unexplored node first, so in the grid it traverses the grid layer by layer. And since the cost of each move in the grid is the same

i.e., 1, then the first path that BFS finds towards food also consists of minimum moves.

Thus BFS algorithm is optimal for this specific grid since all steps have uniform step costs. Moreover, the lecture mentions that BFS is optimal if the cost of each step is 1.

On the other hand, DFS works by expanding the deeper unexplored node first, which means that it keeps moving in one direction until it reaches a point where another option is better than continuing further. Thus it may traverse a very winding way just to get to food although there were some shorter ways as well.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

But if we increase the environment size to a 1000x1000 grid, the number of possible states will become very huge, hence making the memory requirement become one of our concerns.

The space complexity for BFS will be about $O(b^{(d+1)})$ according to the lecture notes. BFS stores many nodes in the memory, especially those in the frontier during the current search depth. So its memory requirement increases exponentially with increasing solution depth, d . Space is considered to be one of the major drawbacks of using BFS as stated in our lectures.

DFS uses relatively less memory compared to BFS. The space complexity for DFS will be $O(bm)$. DFS does not need to store all the nodes at a particular level like BFS does, instead, it just keeps track of the current path along with other alternative paths in the future.

So, in case of huge grid BFS will often face `OutOfMemoryError` since its frontier increases exponentially whereas DFS uses less memory as its requirement is approximately linear. Although, DFS takes too much time exploring deep non-optimal paths and hence cannot guarantee an optimal solution.

UCS faces almost the same issue as BFS does due to the fact that like BFS it stores all those nodes which have been created till now into a priority queue. In UCS all the nodes having path cost lesser than or equal to the cost of optimal solution are considered while calculating its space complexity.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING